Does N-gram Conditional Memory Help Small Dense Transformers? A Minimal Replication of DeepSeek's Engram

Khoa Cao and Kohki Hatori and Haoran Su
Boston University
kcao@bu.edu, khatori@bu.edu, suhr08@bu.edu

Abstract

Large language models must balance compositional reasoning with efficient retrieval of fixed, local patterns such as multi-token entities and formulaic phrases. DeepSeek's Engram introduces conditional n-gram memory for this purpose, but reported gains are entangled with large-scale MoE training and architectural changes, making it unclear whether conditional memory alone helps smaller dense models. We study this question by implementing an end-to-end PyTorch replication on WikiText-103 using a GPT-2 style 85M-parameter decoder-only backbone (Radford et al., 2019). Our framework includes three matched variants: a dense baseline, a parameter-control model with extra dense adapters, and Engram-augmented models with either frozen or learnable gates, all trained under the same tokenizer, data pipeline, optimization schedule, and evaluation protocol. In initial runs, the dense baseline currently achieves the best validation perplexity (28.72), while Engram variants underperform on final perplexity (34.95 and 35.88 for learnable and frozen gates, respectively). However, both Engram variants converge substantially faster (about 34k steps) than the dense baseline (53.7k steps), suggesting a possible optimization or parameter-efficiency advantage despite weaker end perplexity in this setting. Before the final report, we will complete remaining control runs, run test-set evaluation for all variants, and perform diagnostic analyses on various language modeling tasks and visualize gate activations to isolate when conditional memory is beneficial to small dense transformers.

1 Introduction

1.1 Why This Problem Matters

Language modeling involves two qualitatively different subproblems: reasoning about the compositional structure of language and knowledge retrieval. While compositional reasoning requires deep, dynamic computation — something standard transformer architectures are well-suited for — knowledge retrieval — multi- token named entities, formulaic phrases, and other local n-gram regularities, is often more static. Classical n-gram models capture these patterns cheaply via lookup tables, but modern neural LMs discard them entirely, forcing the network to reconstruct static associations through layers of attention and feed-forward computation.

DeepSeek's Engram paper (Cheng et al., 2026) proposes conditional memory as a complementary axis of model capacity: a trainable, hash-addressed n-gram embedding module that is inserted into specific transformer layers and trained jointly from scratch with the backbone. It relies on sparse lookup operations to retrieve static embeddings for fixed-knowledge. The Engram module, specifically, is conditional memory grounded in the classic n-gram structure but with modern additions such as tokenizer compression, multi-head hashing, and contextual gating.

The paper found that an Engram model, when compared to an equally sized dense transformer, achieved better performance on knowledge retrieval, general reasoning, and code/math tasks. However, the gains reported coincided with other architectural innovations such as manifold-constrained hyper-connections - Deepseek's modified constrained residual connection structure that consists of multiple parallel branches that fixes the instability problem of unconstrained hyper-connections (Xie et al., 2026), MoE routing, and a massive scale (27B parameters). It is thus unclear how much of the improvement can be attributed to the Engram module itself, and whether it would still be effective in smaller models.

This project aims to answer the question: Does n-gram conditional memory alone benefit a compact, standard dense transformer trained from scratch?

1.2 Why It Is Difficult

Answering this question comes with a few engineering challenges. First, we must adapt forward pass demonstration in the public Engram repository to a full training loop for a 85M parameter GPT-2 style transformer. We also simplify the multi-branch residual stream, which prior work has shown to be a key driver of Engram's performance (Larsson et al., 2017; Szegedy et al., 2014; Xie et al., 2026; Zhu et al., 2025), to a single branch. Finally, we must reproduce the paper's tokenizer compression technique, which was adapted to the DeepSeek 128k tokenizer, to work for GPT-2's 50k tokenizer.

1.3 Problem Setup and Example

We train four models: a 85M parameter GPT-2 style transformer (Radford et al., 2019), a transformer with an extra dense MLP layer to match the later Engram models' parameter count, a transformer with an Engram module with gating fixed at $\alpha_t = 0.5$ (removing the contextual gating), and a transformer with an Engram module with learnable gating. We train and evaluate the perplexity of these models on the WikiText-103 dataset (Merity et al., 2016).

The key comparison is between the second and fourth models: if the Engram module provides a unique benefit beyond just adding more parameters, then we should see a significant perplexity improvement in the fourth model compared to the second, even though they have the same parameter count. After an initial training run, we find that the ParamsControlLM achieved the lowest training loss and validation perplexity at 3.1979 and 28.1504, respectively. It is followed by the BaselineLM (3.2117 training loss, 28.7229 validation PPL), the EngramLM with learnable gate (3.3184 training loss, 34.9481 validation PPL), and the EngramLM with frozen gate (3.2651 training loss, 35.8792 validation PPL). Even though the Engram models performed slightly worse than the dense baseline, they train signinificantly faster, with the learnable-gate variant converging in 34,000 steps and the frozen-gate variant converging in 34,200 steps, compared to 53,700 steps for the dense baseline and 52,500 steps for the parameter-control baseline. This suggests that the Engram module may provide be more parameter-efficient for learning, even if it does not achieve the lowest perplexity in this setting. We will perform further analyses to understand the training dynamics and the contribution of the Engram module.

2 Progress So Far (Empirical Project)

We have implemented an end-to-end training pipeline in PyTorch, including dataset ingestion, three model variants, optimization with warmup/decay scheduling, periodic validation, and checkpoint resume/save logic. The current codebase supports direct comparison between dense and memory-augmented architectures under matched training conditions.

2.1 Methods

Our implementation consists of (i) a shared GPT-2 style decoder-only backbone, (ii) model-specific adapter modules, and (iii) a unified causal language-model training objective (Radford et al., 2019).

Data ingestion and preprocessing. We load `wikitext-103-raw-v1` via HuggingFace `datasets` and tokenize with the GPT-2 tokenizer (`AutoTokenizer`, fast mode). If no pad token is defined, we set pad to EOS for consistent batching. After tokenization, we concatenate token streams within each split and chunk into fixed-length sequences of length 512. Each chunk is used as `input_ids`, and `labels` are copied from `input_ids`. The actual next-token shift is applied inside the loss function.

Backbone architecture. All model variants share the same backbone defined in `modeling.py`: token and positional embeddings, 12 transformer blocks, RMSNorm pre-normalization, causal multi-head self-attention, and a 2-layer GELU MLP in each block. The language-model head is weight-tied with token embeddings. The training loss is standard teacher-forced causal cross-entropy, implemented by aligning $\texttt{logits}[:, :-1]$ with $\texttt{labels}[:, 1:]$.

Comparing model variants. We currently train three variants from the same base configuration:

- **BaselineLM**: shared dense transformer backbone only.
- **ParamsControlLM**: adds a residual `ControlAdapter` (RMSNorm → up-projection → GELU → down-projection) at selected layers to provide a parameter-count control condition.
- **EngramLM**: adds an `EngramAdapter` at the same selected layers.

Engram adapter implementation. Our Engram module uses hash-addressed n-gram memory with orders $(2, 3)$ inserted at layers $(2, 6)$. For each layer/order/head, we allocate a separate embedding table with prime table sizes to reduce collisions. Given token IDs, we construct shifted n-gram contexts, compute XOR-style hash mixes with layer-specific odd multipliers, and retrieve per-head memory embeddings. Retrieved memory is concatenated and projected to key/value vectors. The query comes from current hidden states. A scalar gate per token is either learnable ($\sigma((q \cdot k)/\sqrt{d})$) or fixed at 0.5 for ablation. The final memory update includes a depthwise causal short-convolution branch and is added residually to the block output.

Training loop status. We have implemented a full optimizer-step loop with gradient accumulation, gradient clipping, scheduler stepping, periodic validation perplexity, CSV logging, checkpoint save/resume, and final test-set perplexity export. This infrastructure is complete and currently used for all model variants.

2.2 Experiments

Dataset and splits. All experiments use WikiText-103 with the standard train/validation/test splits provided by HuggingFace (Merity et al., 2016). Preprocessing follows the fixed-length chunking procedure described above (block size 512), with no additional corpus filtering beyond tokenizer processing.

Evaluation metric. Our primary metric is perplexity (PPL), computed as $\exp(mean(L_{CE}))$ on validation and test batches. Evaluation uses the same shifted causal objective as training. During training, validation PPL is measured every 5,000 optimizer steps; final reporting includes test PPL (up to 500 test batches in the current script).

Training configuration. Current default settings are: batch size 8, gradient accumulation 4 (effective batch size 32), 100,000 optimizer steps, AdamW ($\texttt{lr} = 3 \times 10^{-4}$, weight decay 0.1), linear warmup for 2,000 steps, cosine decay to 10^{-5}, and gradient clipping at 1.0. We train on CUDA when available (otherwise CPU), use deterministic seeds for Python/NumPy/PyTorch, and log step/loss/PPL/LR to per-model CSV files. Checkpoints are saved every 10,000 steps and can be resumed from disk.

Comparison protocol. The active comparison includes:

- **Dense baseline** (BaselineLM),
- **Parameter-control baseline** (ParamsControlLM), and
- **Engram memory model** (EngramLM), with both frozen-gate and learnable-gate settings.

All variants share tokenizer, data pipeline, sequence length, optimization schedule, and evaluation procedure to isolate effects of memory augmentation versus added dense parameters.

2.3 Results

The results of the initial training runs are summarized in Table 1. Figures 1 and 2 show the training loss and validation perplexity for each model variant across the training process. The ParamsControlLM achieved the lowest training loss and validation perplexity, followed by the BaselineLM, the EngramLM with learnable gate, and the EngramLM with frozen gate. However, the Engram models converged significantly faster than the dense baseline, suggesting that the Engram module may provide more parameter-efficient learning even if it does not achieve the lowest perplexity in this setting.

Figure 1: Training loss across steps for each model variant.

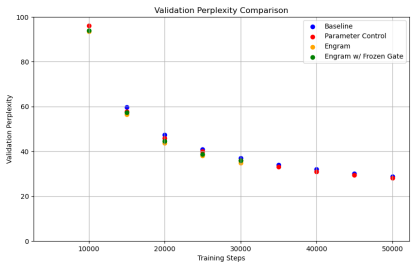

Figure 2: Validation perplexity across steps for each model variant.

With that said, the current results are suprising and we will perform further checks to confirm that

Model Variant	Lowest Training Loss	Lowest Validation PPL	Steps to Converge
BaselineLM	3.2117	28.7229	53,700
ParamsControlLM	3.1979	28.1504	52,500
EngramLM (Learnable Gate)	3.3184	34.9481	34,000
EngramLM (Frozen Gate)	3.2651	35.8792	34,200

Table 1: Training loss, validation perplexity, and convergence steps for each model variant.

the training loop and Engram module are implemented correctly, and that the hyperparameters are appropriate for all variants. We will also analyze the training dynamics to understand why the Engram models converge faster but to a higher perplexity than the dense baseline.

3 Project Plan (Until Final Report)

With the training loop implemented and initial experiments underway, our next steps are to perform final test-set evaluation and perform diagnostic analyses to understand the contribution of the Engram module versus added parameters. For diagnostic analyses, we plan to compare the **ParamsControlLM** and **EngramLM** (both frozen and learnable gate variants) on several tasks that require different degrees of knowledge retrieval and compositional reasoning, such as named-entity prediction and formulaic phrase completion. We will also visualize the gating scalar α_t using heatmaps to confirm activation patterns similar to those in Figure 7 of (Cheng et al., 2026), that is, high activations for tokens that complete n-gram patterns.

3.1 Planned Milestones

- Milestone 1 (4/21/2026): Complete training runs for all model variants and report validation & test perplexities.
- Milestone 2 (4/29/2026): Perform diagnostic analyses comparing ParamsControlLM and EngramLM on knowledge retrieval vs compositional reasoning tasks.
- Milestone 3 (5/5/2026): Visualize gating scalar activations and compare to expected patterns.
- Milestone 4 (5/7/2026): Finalize report with results, analyses, and discussion.

References

Xin Cheng, Wangding Zeng, Damai Dai, Qinyu Chen, Bingxuan Wang, Zhenda Xie, Kezhao Huang, Xingkai Yu, Zhewen Hao, Yukun Li, Han Zhang, Huishuai Zhang, Dongyan Zhao, and Wenfeng Liang. 2026. Conditional memory via scalable lookup: A new axis of sparsity for large language models. *arXiv preprint arXiv:2601.07372*.

Gustav Larsson, Michael Maire, and Gregory Shakhnarovich. 2017. Fractalnet: Ultra-deep neural networks without residuals. In *International Conference on Learning Representations*.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer sentinel mixture models. *Preprint*, arXiv:1609.07843.

Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners.

Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. 2014. Going deeper with convolutions. *Preprint*, arXiv:1409.4842.

Zhenda Xie, Yixuan Wei, Huanqi Cao, Chenggang Zhao, Chengqi Deng, Jiashi Li, Damai Dai, Huazuo Gao, Jiang Chang, Kuai Yu, Liang Zhao, Shangyan Zhou, Zhean Xu, Zhengyan Zhang, Wangding Zeng, Shengding Hu, Yuqing Wang, Jingyang Yuan, Lean Wang, and Wenfeng Liang. 2026. mhc: Manifold-constrained hyper-connections. *Preprint*, arXiv:2512.24880.

Defa Zhu, Hongzhi Huang, Zihao Huang, Yutao Zeng, Yunyao Mao, Banggu Wu, Qiyang Min, and Xun Zhou. 2025. Hyper-connections. In *The Thirteenth International Conference on Learning Representations*.